

A-DMFE — Technical & Codebase Report

Adaptive Dynamic Multi-Service Feasibility Engine — tech stack, architecture & code logic, dashboard walkthrough, unwanted-file audit, and optimization recommendations.

A-DMFE unifies ride-hailing, food delivery and parcel delivery into one dispatch platform. Its core is the Dynamic Multi-Service Feasibility Engine (DMFE): a compatibility-scoring, batching and Google OR-Tools routing pipeline that groups compatible requests into shared trips, plus an Adaptive layer (A-DMFE) that tunes the engine's weights and thresholds to live traffic, demand and fleet conditions, and an Explainable-AI (XAI) layer that exposes the reasoning behind every decision. The frontend is a React 19 admin console with a shared map engine (Google Maps / Leaflet) driving three live dashboards: Overview, Live Operations and AI Insights.

1. Technology Stack

1.1 Frontend

Library	Version	Role in this project
React	19.2.7	UI framework — all pages/components; routes lazy-loaded with Suspense
Vite	8.1.1	Dev server & build tool (rolldown-based bundler)
React Router DOM	7.18.1	Client-side routing (App.jsx route table)
Tailwind CSS	4.3.3	Utility-first styling; custom glass-panel design system in index.css
@react-google-maps/api	2.20.8	Google Maps rendering when VITE_GOOGLE_MAPS_API_KEY is set
react-leaflet / leaflet	5.0.0 / 1.9.4	Keyless OpenStreetMap fallback map when no Google key is present
Recharts	3.10.0	Charts — score distribution, decision breakdown, analytics
Framer Motion	13.0.0	Panel transitions, animated KPI counters
Axios	1.18.1	HTTP client (services/api.js) with baseURL + short GET cache/dedup
lucide-react	1.24.0	Icon set used across every dashboard
react-hot-toast	2.6.0	Toast notifications (login errors, simulation actions)
oxlint	1.71.0	Frontend linter (npm run lint) — not ESLint

Vitest / Playwright / @testing-library are **not installed** — src/setupTests.js and the vite.config.js test block are scaffolding for future frontend tests, currently unused.

1.2 Backend

Library	Version	Role in this project
FastAPI	>=0.115	REST API framework — all /api routers in app/api/routes/
Uvicorn	>=0.30	ASGI server
SQLAlchemy	>=2.0.36	ORM — models in app/db/models.py
SQLite / PostgreSQL	n/a	Dev DB: dmfe_dev.db (SQLite, auto-created). Prod: PostgreSQL via DATABASE_URL

Library	Version	Role in this project
Google OR-Tools	>=9.10	Pickup-and-Delivery VRP solver — app/dmfe/optimizer.py
Pydantic / pydantic-settings	>=2.10 / >=2.6	Request/response schemas + .env-driven Settings
python-jose	>=3.3 (cryptography)	JWT access tokens
bcrypt	>=4.0	Password hashing (raw bcrypt, not passlib)
slowapi	>=0.1.9	Rate limiting
pytest / httpx	>=8.0 / >=0.27	Backend test suite (backend/tests/)

The project README's architecture diagram lists a WebSocket server for live tracking, but no WebSocket route, dependency, or client exists anywhere in the codebase — live updates are actually done by REST polling (the frontend hooks re-fetch on an interval; see §3.4). This is a documentation/implementation mismatch worth fixing before it is shown to evaluators — either implement the WebSocket path or correct the README and architecture diagram to describe polling.

2. Architecture & Code Logic

2.1 End-to-end request flow

Every pending ride/food/parcel request moves through one pipeline, orchestrated by `app/dmfe/pipeline.py` (also reachable stage-by-stage via the individual engine modules for analysis-only calls):

- **Compatibility scoring** (`compatibility.py` / `scoring.py` / `score_engine.py`) — five pure, DB-free factor functions (pickup distance, route/destination overlap, time-window fit, vehicle capacity, priority) combine into a weighted Compatibility Score $CS = w1 \cdot \text{Pickup} + w2 \cdot \text{Route} + w3 \cdot \text{Time} + w4 \cdot \text{Capacity} + w5 \cdot \text{Priority}$, each factor in $[0,1]$.
- **Batch generation** (`batch_generator.py`) — all pairwise candidates within a pickup radius are scored; cheap pre-checks (time window, radius, capacity) run before the full 5-factor evaluation to skip unnecessary work, then a greedy disjoint assignment picks the best non-overlapping pairs.
- **Decision Engine — Gate A-E** (`decision_engine.py`) — a candidate group must clear five gates: A) Compatibility Score $\geq$ threshold, B) vehicle capacity sufficient, C) time compatibility acceptable, D) priority rules (High priority never delayed for a lower-priority match), E) driver availability. All five passing $\rightarrow$ persisted as a Shared Trip (`decision = Compatible`); otherwise the requests dispatch individually.
- **Route optimization** (`optimizer.py`) — Google OR-Tools solves a Pickup-and-Delivery VRP for the batch, minimizing a multi-objective arc cost ($\text{distance} + w_{\text{time}} \cdot \text{duration} + w_{\text{fuel}} \cdot \text{fuel_cost}$) under hard constraints (capacity, pickup-before-drop, max delay) and a soft priority bonus. Distance/time comes from the Google Distance Matrix API when a key is configured, otherwise a haversine fallback.
- **Driver selection** (`driver_selection.py`) — scores available, non-busy drivers by proximity/ETA and vehicle-type/capacity fit, then commits Trip + DriverAssignment rows atomically, marking the driver/vehicle Busy and the requests Assigned.

2.2 The Adaptive layer — A-DMFE (`app/dmfe/adaptive/`)

On top of the fixed Phase-9 pipeline above, eight adaptive modules replace static thresholds/weights with values derived from live system state. Every module degrades to the exact static (Phase 9) behaviour when `admfe.mode = "static"`, so the adaptive layer is provably backward-compatible rather than a parallel implementation.

Module	File	What it does
1. Context Awareness	<code>adaptive/context.py</code>	Builds a normalized ContextProfile each run — traffic density, demand pressure, driver availability, capacity stress, service-mix entropy, time-of-day, priority pressure.
2. Adaptive Weights	<code>adaptive/weights.py</code>	Recomputes the 5-factor weight vector per context (e.g. heavy traffic $\rightarrow$ route weight up, few drivers $\rightarrow$ capacity weight up), normalized to sum to 1.
3. Extension Factors	<code>adaptive/factors.py</code>	Nine extra compatibility dimensions (expected delay, utilization, waiting time, driver workload, historical success, environmental impact...) feeding the Batch Quality Score, never the core CS.
4. Compatibility Matrix	<code>adaptive/matrix.py</code>	Builds the full pairwise CS matrix for the pending pool once per run; spatial bucketing (latitude bands) cuts the naive $O(n^2)$ pass to $O(n \cdot k)$.
5. Intelligent Batch Formation	<code>adaptive/batching.py</code>	Forms batches passing BOTH $CS \geq \text{theta_eff}$ and Batch Quality Score $\geq \text{theta_bqs}$, then expands strong pairs into 3-member batches where capacity allows.

Module	File	What it does
6. Adaptive Decision Engine	adaptive/decision.py	Derives theta_eff and theta_bqs from the ContextProfile (linear formulas clamped to configured bounds) and a decision-confidence score from score margin, factor agreement and delay headroom.
7. Explainable AI (XAI)	adaptive/xai.py	Signed factor attribution $a_f = w_f \cdot (f_f - 0.5)$ against a neutral 0.5 baseline, ranked by magnitude — this is what powers the AI Insights dashboard's reason list.
8. Learning	adaptive/learning.py	Closed-loop correction: logs predicted-vs-actual residuals (delay, fuel, utilization) per corridor into ring buffers, refitting per-corridor multipliers every N completed trips.

2.3 API layer (FastAPI routers)

Prefix	Router file	Serves
/api/auth	auth.py	Login, profile, logout (JWT)
/api/providers	providers.py	Provider CRUD (ride/food/parcel operator configuration)
/api/dashboard	dashboard.py	Aggregate stats consumed by the Overview page
/api/orchestration	orchestration.py	Legacy AI orchestration endpoints (superseded by DMFE — see §4)
/api/simulation	simulation.py	Simulation engine start/pause/stop, live queue
/api/xai	xai.py	XAI overview + explanations feeding the AI Insights map/cards
/api/notifications	notifications.py	Activity timeline / notification center
/api/vehicles, /api/drivers	drivers.py	Fleet + driver CRUD, live locations, stats
(self-prefixed)	config.py	System configuration, AI rules, audit log, backup/restore
(self-prefixed)	playback.py	Scenario playback / comparison
/api/dmfe (v2)	dmfe_v2.py	Statistics, batches, demo seed/clear
/api/dmfe (engine)	dmfe_engine.py	Trips, queue, context, assignments — the live DMFE dispatch surface
/api/health	main.py	Bare health check

2.4 Frontend architecture

- **Routing / code-splitting** — App.jsx lazy-loads every page behind React Router; a ProtectedRoute gate wraps the whole admin shell (AdminLayout) except /login.
- **services/api.js** — single Axios instance (VITE_API_URL, default localhost:8000/api) with a short in-memory GET cache/dedup window so simultaneous pollers don't fire duplicate requests.
- **Live data via polling, not WebSockets** — hooks/useOperationalNetwork.js polls vehicle locations, vehicle stats, active trips and the pending queue on an interval (5-8s depending on page) and is shared by both the Overview and Live Operations pages so their numbers can never disagree; the AI Insights map polls the simulation queue every 2.5s.
- **utils/requestSemantics.js** — single source of truth for every colour/icon/label pairing (request type, vehicle status, decision state, route colours) so the same request type is never drawn differently on two dashboards.
- **components/map/** — one shared map engine, LiveMapContainer.jsx, parameterized by a mode prop (network / operations / decision) so Overview, Live Operations and AI Insights render through the same Google Maps + Leaflet-fallback code rather than three separate map implementations. It composes

FleetLayer (clustered vehicle markers), ActiveTripsLayer (background corridors), RequestMarkers (pending queue) and XAIHighlightLayer (the selected decision's route).

- **utils/routeUtils.js** — road-following geometry for the map: Google Directions -> OSRM public API -> straight-line, in that order, cached by coordinate fingerprint, with OR-Tools' stop order always passed through unmodified (`optimizeWaypoints: false`) so the map never silently re-solves the backend's routing decision.
- **utils/operationalRoute.js** — reconstructs the authoritative stop sequence (depot -> OR-Tools `stop_order`) from a normalized XAI/trip payload; this is the single place stop order is assembled for display, reused by both the map layer and the detail panels so the two can never disagree.

3. Dashboard Details

3.1 Overview (/dashboard)

A pure data/analytics dashboard — no map. Every figure is sourced from a real backend endpoint (dashboard/stats, dmfe/statistics, dmfe/trips, notifications/timeline, xai/overview, simulation/status, health), aggregated client-side; nothing is invented.

- Eight top KPI cards: Active Requests, Shared Trips, Individual Trips, Vehicles, Vehicle Utilization %, Fuel Saved (L), CO₂ Reduction (kg), Completed Trips.
- Service Demand / Fleet Status / Trip Status — three progress-bar panels broken down by request type, vehicle status and trip status respectively.
- Recent Activity (shared ActivityTimeline component) and Top Performing Vehicles (ranked by trip count, then distance saved).
- AI Insights summary tile (decisions evaluated, average compatibility/confidence, most common outcome) and a System Health panel (API / live-feed / simulation-engine status).

3.2 Live Operations (/live-map)

The full-detail operational map — every vehicle, every pending request, every active corridor, with a professional dark-navy Google/Leaflet basemap (roads deliberately lifted to a legible slate against the navy ground so a route can be checked against the street it claims to follow).

- TOP-CENTER: floating KpiBar (trips in flight, vehicles deployed, per-type queue counts) plus engine start/pause/stop/clear controls.
- LEFT: collapsible MapFilterPanel (search, type/provider/priority filters).
- CENTER: the map itself — clustered fleet markers, background corridor polylines, pending-request markers, and (via ?xai= deep link) the same XAI highlight layer used on AI Insights.
- RIGHT: TripDetailsPanel — inspector for whatever is selected (a queued request, a vehicle, or a deep-linked XAI decision), only mounted when something is actually selected.
- BOTTOM: ActivityBar — a horizontal strip of the most recent queue items, clickable to select on the map.

3.3 AI Insights / Explainable Decisions (/xai)

Just redesigned in this session to a literal LEFT / CENTER / RIGHT layout matching Live Operations' visual language, replacing an earlier version that floated all decision detail on top of the map.

- Above the fold (unchanged): summary cards, a score-distribution bar chart, a compatibility gauge and a decision-breakdown pie chart, all driven by /api/xai/explanations.
- LEFT: the Decision Explanation Cards list (DecisionCard.jsx) plus a factor-breakdown chart for the selected decision.
- CENTER: XaiMapPanel — the shared map engine in "decision" mode, now trimmed to one compact identity strip (request id, type, decision, score) instead of the large floating panels it used to carry.
- RIGHT (new): XaiDecisionPanel — classification (Batch/Individual/Rejected), decision + reason, compatibility/confidence, driver/vehicle, pickup/drop, the OR-Tools stop order as a numbered list ("1. Pickup Request #..."), related requests, factor values and key reasons.
- Rejected decisions now draw no route line on the map (a real gap fixed this session — previously every un-dispatched request, rejected or not, drew the same dashed "planned" line) and their stop markers carry a red rejection indicator instead of a sequence badge.

3.4 Other admin pages

Route	Page	Purpose
/providers	ProviderManagement.jsx	CRUD for ride/food/parcel provider operators
/datasets	DatasetManagement.jsx	Upload/manage datasets used to seed simulations
/drivers	DriverDashboard.jsx	Driver + vehicle roster, assignment history, live locations
/simulation-monitor	SimulationMonitoring.jsx	Simulation engine controls and live monitoring
/analytics	AnalyticsDashboard.jsx	Deep analytics — separate from Overview's high-level KPIs
/dmfe	DMFEDashboard.jsx	DMFE batches, candidate groups, pending/rejected queues
/playback	ScenarioDashboard.jsx	Saved-simulation replay and scenario comparison
/config	SystemConfiguration.jsx	AI rules, weights/thresholds, audit log, backup/restore
/notifications	NotificationCenter.jsx	Full notification/activity center
/ai-orchestration	AIDashboard.jsx	Legacy pre-DMFE orchestration view — see §4 (dependent on a dead backend route)

4. Unwanted Files & Dead Code

This is a snapshot of what is physically present in the repository today. It separates three different things that are easy to lump together but need different treatment: dead application code (safe to delete, nothing imports it), throwaway/regenerable artifacts (safe to delete, they're either git-ignored already or trivially regenerated), and the project's own academic paper/report deliverables (not "unwanted" — just recommended to move out of the application repository so they stop inflating its size and confusing a code reviewer).

4.1 Confirmed dead application code

Path	Status	Evidence
backend/app/engine/optimizer.py	Already deleted	Only its compiled bytecode (optimizer.cpython-314.pyc) still sits in engine/ __pycache__ / — the .py source is gone. This was the legacy pre-DMFE "AIOrchestrator" VRP solver, superseded by app/dmfe/optimizer.py.
backend/app/engine/explainability.py	Already deleted	Same as above — only explainability.cpython-314.pyc remains. Zero live imports; superseded by app/dmfe/adaptive/xai.py.
frontend/src/old_livemap.jsx (repo root)	Dead, still present (21.8 KB)	Sits at the repo root, not under frontend/src, so it is not even reachable by an import — a leftover snapshot of LiveMapContainer.jsx from before the road-routing/redesign work.
frontend/old_xai_layer.jsx (repo root)	Dead, still present (0 bytes)	Empty placeholder at the repo root, not under frontend/src.
frontend/src/pages/AIDashboard.jsx	Live route, dead backend dependency	Still routed at /ai-orchestration and still imported, but it talks to the legacy orchestration.py endpoints tied to the deleted engine/ module above — worth confirming this page still works, or retiring the route along with the legacy router.

The project already ran a self-audit on 2 Sep 2026 (dead_code_audit/ at the repo root) that additionally flagged frontend/src/pages/DashboardOverview.jsx and its dashboard/ KPI components, and components/map/MapFilters.jsx, as unreferenced. DashboardOverview.jsx and its components are confirmed gone today (superseded by the current Dashboard.jsx). MapFilters.jsx was not re-verified live in this session — check it against frontend/src/components/map/ before relying on it.*

4.2 Repository clutter (root level)

The repo root currently holds a large amount of one-off script/report output from earlier work sessions that has no ongoing purpose:

- **Report/paper-generation scripts** — analyze_backend.py, build_base.py, generate_base_docs.py, generate_deep_docs.py, generate_final_docs.py, generate_final_docs_11.py, generate_final_resp.py, scratch_docx.py, scratch_read_camera.py, scratch_read_resp.py, scratch_tex.py — one-shot generators for the .docx/.pdf report files sitting next to them; not part of the running application.
- **Generated report/paper output** — AI_Powered_Unified_Mobility_DMFE_FINAL11.docx/.pdf, AI_Powered_Unified_Mobility_DMFE_FINALPPG.docx/.pdf, Response_to_Reviewers_FINAL111.docx, Response_to_Reviewers_FINALPPG.docx (duplicate FINAL/FINALPPG pairs — pick the actual final one and archive the rest), plus doc_text.txt/doc_text2.txt, tables.txt, tables_all.txt, scratch_read_camera_ready.txt, scratch_read_docx.txt, scratch_read_responses.txt — raw text dumps used while assembling those documents.
- **Prior audit artifacts** — dead_code_audit/, dead_code_audit.zip, dead_code_audit_manifest.csv, pre_audit_sha256.txt, audit_results.json, backend_analysis.json, storage_audit.json — a completed,

one-time code audit; keep the report if useful as a record, but the .zip + manifest + checksums are redundant with it.

- **GEMINI_PROJECT_KNOWLEDGE/ and GEMINI_PROJECT_KNOWLEDGE.zip (4.1 MB)** and GEMINI_PACKAGE_CONTENTS.txt — a documentation package generated for a different AI tool's context; not used by the running app.
- **Stray editor/filesystem temp files** — nine .fuse_hidden0000... files at the repo root and more under backend/ (32 KB each) are FUSE artifacts left behind when a file was deleted while still open elsewhere; already covered by .gitignore's .fuse_hidden* rule but still occupying real disk space until deleted.
- **Misc scratch** — completion_context.txt, commit_message.txt (git-ignored already), xai.json (0 bytes), archive/, temp/ (has its own _to_delete/ subfolder already started), experiments/ (empty), qa/server_err.log + server_out.log (old logs, duplicated by backend/server_err.log and backend/server_out.log — see 4.3).

4.3 Backend runtime clutter

- **dmfe_dev.db-shm / dmfe_dev.db-wal** — SQLite WAL side-files; normal while the server is running, safe to delete when it is stopped (SQLite regenerates them).
- **qa_test.db / -shm / -wal (11 MB)** — a separate test database from a QA run; not used by normal dev startup (config.py's SQLite fallback always targets dmfe_dev.db).
- **server_err.log (45 KB) / server_out.log** — stale logs from a past run; already git-ignored via *.log but worth clearing locally.
- **__pycache__/, .pytest_cache/, .venv/** — all already git-ignored; fine to leave, or purge with `find . -name __pycache__ -exec rm -rf {}` + if disk space is tight.
- **A stray, duplicate dmfe_dev.db (4 KB) also sits at the repo root** (distinct from backend/dmfe_dev.db, which is the real 37 MB active database) — a leftover from an earlier working directory; safe to delete, it is not the database the app actually reads.

None of the paper/, docs/, Claude outputs/ (screenshots) or the two duplicate .docx/.pdf report pairs are "unwanted" in the sense of being safe to delete outright — they are the project's own academic deliverables. The recommendation is only to move them out of the application repository (a separate "submission" or "docs" repo/folder outside rapidoproject/) so the codebase a reviewer or grader clones stays focused on the actual running application.

5. Optimization Recommendations

5.1 Repository hygiene

- Delete the confirmed-dead files in §4.1 (`old_livemap.jsx`, `old_xai_layer.jsx`, and the orphaned `engine/*.pyc` caches) and re-run the existing `dead_code_audit` tooling once, since a chunk of its original findings have already been fixed and the report is now partly stale.
- Move `paper/`, `docs/`, `GEMINI_PROJECT_KNOWLEDGE*`, `dead_code_audit*`, and the root-level `generate_*/scratch_*` scripts and their `.docx/``.pdf/``.txt` output out of `rapidoproject/` into a separate location — this alone removes well over 10 MB of non-application content from the repo a reviewer would clone.
- Add the currently-uncovered clutter to `.gitignore` going forward: `archive/`, `experiments/`, `qa/`, `"Claude outputs/"`, `*.zip` audit bundles, and `*_audit*.json/``.csv` result files, so future one-off sessions stop re-accumulating this.
- Resolve the two duplicate final-report pairs (FINAL11 vs FINALPPG) to a single canonical version before submission — having two "FINAL" documents is itself a common grading red flag.

5.2 Documentation accuracy

- Fix the README's architecture diagram/tech-stack section — it currently claims a WebSocket server for live tracking, but the real implementation is REST polling (see §1.2/§3.4). An evaluator who reads the README and then inspects the code will find the mismatch immediately.
- The README also lists `websocket`-style live tracking as a completed feature under "Security Hardening Report (Phase 9)" framing that reads as an external audit; consider clarifying these are self-reported development notes rather than a third-party audit, to avoid an awkward question in a viva.

5.3 Frontend

- Either wire up the Vitest scaffolding that already exists (`setupTests.js`, the `vite.config.js` test block) with real component tests, or remove the scaffolding — right now it signals "tests exist" without any actually running.
- `AlDashboard.jsx` (`/ai-orchestration`) depends on the legacy `orchestration.py` router tied to the now-deleted `engine/` module (§4.1) — confirm it still renders correctly, or retire the route together with the legacy backend endpoints it calls, rather than leaving a partially-dead page reachable from the nav.
- The XAI dashboard polls every 2.5s and Live Operations every 5-8s; both already share `useOperationalNetwork` so they never issue duplicate requests for the same data — no change needed there, but if the demo dataset grows much larger, lengthening the AI Insights poll interval (it is the tightest one in the app) is the cheapest first lever before touching any query.

5.4 Backend

- `app/dmfe/adaptive/matrix.py` already does the important optimization here — spatial bucketing to avoid the naive $O(n^2)$ pairwise compatibility pass; no further algorithmic change is indicated by anything reviewed for this report.
- The 37 MB dev SQLite database plus WAL/SHM files, and the unused 11 MB `qa_test.db`, add up to real disk footprint in a student environment — periodic cleanup (or a `scripts/reset_db.sh`) keeps local dev light.
- Remove the stray root-level `dmfe_dev.db` (4 KB, §4.3) so nobody ever accidentally points a tool at it instead of `backend/dmfe_dev.db`.

Generated from a live read of the `rapidoproject` repository (backend/app, frontend/src, repo root and backend root directory listings, `package.json`, `requirements.txt`, `main.py`, `App.jsx`, and the module docstrings of every DMFE/A-DMFE engine file). Where a finding could not be re-verified live in this session (the device bridge disconnected partway through the audit), it is

marked as sourced from the project's own prior self-audit rather than independently confirmed.